

Java – Beautifully Broken

You left at Java 11. Here's everything you missed.

Issue #1

Thu, 01 Oct 2026

🕒 ~20 min read

Level 3/5

⚡ The short version

1. **Java ships every six months now, with an LTS every two years.** 8 → 11 → 17 → 21 → 25 are the LTS stops, and the next one is **Java 29 in September 2027**.
2. **The real upgrade is in how you model data and run concurrent work:** records + sealed types + pattern matching, and virtual threads. Part 1 maps it all in one sitting.
3. **Java 27 landed on 15 September** with G1 as the default collector everywhere and compact object headers switched on by default, a free memory win. Spring Boot 4.2 is at its second milestone.

Version watch

25

current **LTS** (Sep 2025),
the safe target for
production

27

latest JDK, **GA 15 Sep
2026** (non-LTS)

4.1.1

latest **Spring Boot** GA
(20 Aug 2026); 4.2 is in
milestones

29

next planned **LTS**,
September 2027

Sources: Oracle Java SE support roadmap, Oracle, Java 27 release, spring.io.

PART 1 Learn

🗺 The map: Java 8 → 27, and what actually matters

CATCH-UP

~10 MIN

LEVEL 3

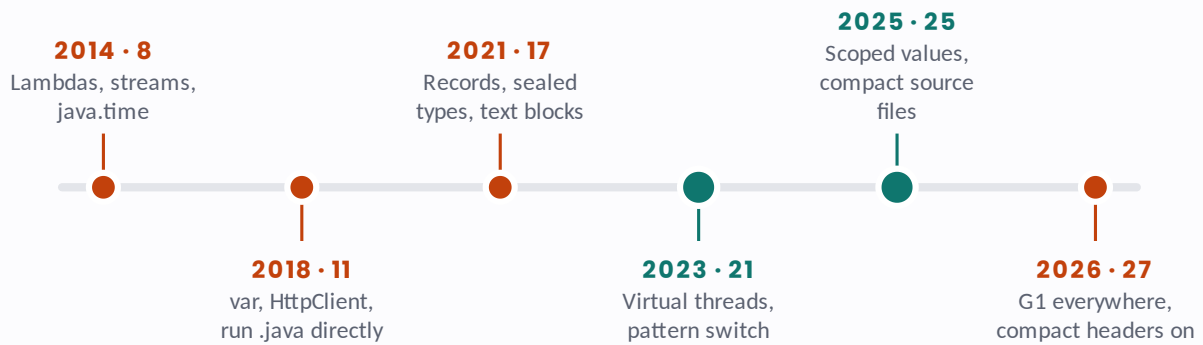
If your mental model of Java is "Java 8 with Spring," you're not behind on syntax. You're behind on a whole way of writing Java. Nineteen releases have shipped since Java 8. Most changes are small, but a handful change how you'd design a service from scratch. Even Java 17 is no longer new: Oracle's Premier Support for it ended in **September 2026**. (Oracle support roadmap)

Today is the map. The next few issues zoom into each region.

FIRST, HOW JAVA SHIPS NOW

Since Java 10, there's been a **feature release every six months** (March and September) and a **long-term support (LTS)** release every two years. Most companies run only LTS versions in production. Non-LTS releases are where features mature. (Oracle support roadmap)

The LTS stops (plus today's release)



Java 27 is not an LTS. It's shown because it's today's latest. Sources: OpenJDK release pages for 17, 21, 25; InfoQ on Java 27.

Jargon buster

- **JEP** (JDK Enhancement Proposal): the design document behind each feature. "JEP 444" is how Java people refer to virtual threads.
- **Preview feature**: fully built but not final. It's off unless you pass `--enable-preview`, and it may change. Don't ship previews to production.
- **Incubator**: an experimental API in a `jdk.incubator.*` module, one step before preview.
- **LTS**: a release that vendors patch for years. Non-LTS releases get about six months of updates.

THE FEATURES THAT MATTER, BY WHEN THEY BECAME FINAL

Feature	Final in	JEP	Why you'd care
<code>var</code> for local variables	10	286	Less noise; the type is still static
Standard <code>HttpClient</code>	11	321	HTTP/2 and async, with no extra library
Switch expressions (<code>-></code> , <code>yield</code>)	14	361	<code>switch</code> returns a value, with no fall-through bugs
Text blocks (<code>"""</code>)	15	378	Readable SQL/JSON in code
Records	16	395	Immutable data classes in one line
Pattern matching for <code>instanceof</code>	16	394	No more cast-after-check
Sealed classes	17	409	Closed hierarchies the compiler understands
Record patterns + pattern matching for <code>switch</code>	21	440 / 441	Deconstruct data and branch on shape safely
Virtual threads	21	444	Millions of cheap threads for blocking I/O
Sequenced collections	21	431	<code>getFirst()</code> / <code>getLast()</code> / <code>reversed()</code> everywhere
Unnamed variables <code>_</code>	22	456	Say "I don't need this" explicitly

Feature	Final in	JEP	Why you'd care
Stream gatherers	24	485	Custom intermediate stream operations
Scoped values	25	506	A safer <code>ThreadLocal</code> for the virtual-thread era
Compact source files, instance <code>main</code>	25	512	Java as a scripting language

The rows from 17 and 21 are confirmed on the OpenJDK release pages. The full list, release by release, is in Oracle's [Java Language Changes by Release](#). [\(OpenJDK 17\)](#) [\(OpenJDK 21\)](#) [\(OpenJDK 25\)](#)

BIG IDEA #1: DATA-ORIENTED JAVA

Records, sealed types and pattern matching are three features designed to work as one. Together they let you say "a Shape is exactly a Circle or a Square, nothing else", and the compiler holds you to it.

Same logic, two eras

Java 8: classes + instanceof chains

```
abstract class Shape {}

final class Circle extends Shape {
    final double r;

    Circle(double r) { this.r = r; }

    // + equals, hashCode, toString...
}

// Square: same again...

double area(Shape s) {
    if (s instanceof Circle) {
        Circle c = (Circle) s;
        return Math.PI * c.r * c.r;
    } else if (s instanceof Square) { ... }
    throw new IllegalArgumentException();
}
```

Java 21+: sealed + records + switch

```
sealed interface Shape
    permits Circle, Square {}

record Circle(double r)
    implements Shape {}

record Square(double side)
    implements Shape {}

double area(Shape s) {
    return switch (s) {
        case Circle c ->
            Math.PI * c.r() * c.r();
        case Square q ->
            q.side() * q.side();
    }; // no default needed: the
} // compiler knows every case
```

Add a `Triangle` to the sealed interface and every `switch` that doesn't handle it **stops compiling**. The Java 8 version would fail at runtime, in production.

That's the payoff: **exhaustiveness checking**. In a payments service, `sealed interface PaymentResult permits Approved, Declined, NeedsOtp` means nobody can forget the OTP branch. Records give you `equals`, `hashCode`, `toString` and accessors for free. Their fields are final, so they're a natural fit for DTOs, events and value types.

BIG IDEA #2: CONCURRENCY GOT CHEAP AGAIN

For 20 years, a Java thread was a thin wrapper over an OS thread: about a megabyte of stack, and expensive to create and to switch between. So we pooled them, and when pools ran dry we reached for reactive frameworks.

Virtual threads (Java 21) are managed by the JVM. When one blocks on I/O, the JVM parks it and frees the OS thread underneath for other work.

What runs where with virtual threads

Your code, one virtual thread per request

millions possible, cheap to block

JVM scheduler (a ForkJoinPool)

mounts/unmounts virtual threads

Carrier threads (platform threads)

≈ one per CPU core

OS threads & CPU cores

what actually executes

Calling a slow API for 10,000 orders

Java 8: a bounded pool of OS threads

```
ExecutorService pool =  
    Executors.newFixedThreadPool(200);  
for (Order o : orders) {  
    pool.submit(() -> callPaymentApi(o));  
}  
  
// 200 run at once; the other  
// 9,800 wait in the queue
```

Java 21+: one virtual thread per task

```
try (var exec = Executors  
    .newVirtualThreadPerTaskExecutor()) {  
    for (Order o : orders) {  
        exec.submit(() -> callPaymentApi(o));  
    }  
} // close() waits for all tasks  
// all 10,000 can wait on I/O at once
```

Illustrative. Virtual threads help **I/O-bound** work (HTTP, JDBC, queues), not CPU-bound number crunching. The real limit moves to what you call: the payment API and your connection pool still need explicit limits.

BIG IDEA #3: THE SMALL WINS YOU'LL USE EVERY DAY

None of these will change your architecture, but together they remove a surprising amount of noise from everyday code.

Everyday code, then and now

Java 8

```
Map<String, List<Order>> byCity =
    new HashMap<String, List<Order>>();

String sql = "SELECT id, total\n" +
    "FROM orders\n" +
    "WHERE city = ?";

String label;
switch (status) {
    case NEW: label = "Pending"; break;
    case PAID: label = "Done"; break;
    default: label = "?";
}

Order last = list.get(list.size() - 1);
```

Java 21+

```
var byCity =
    new HashMap<String, List<Order>>();

String sql = """
    SELECT id, total
    FROM orders
    WHERE city = ?
    """;

String label = switch (status) {
    case NEW -> "Pending";
    case PAID -> "Done";
    default -> "?";
};

Order last = list.getLast();
```

`var` (10), text blocks (15), switch expressions (14) and `getLast()` from sequenced collections (21). A forgotten `break` in the old switch silently falls through; the arrow form can't.

A few rules of thumb from teams that adopted these:

- Use `var` when the right-hand side already says the type (`new` , factory methods). Avoid it when the type is the point, like `var result = service.process(x);` .
- Text blocks strip common indentation for you, so keep the closing `"""` on its own line, aligned with the content. Pair them with `.formatted(...)` for simple templating.
- Switch expressions must be exhaustive. With enums, leaving out `default` means a new enum constant becomes a compile error, the same safety net sealed types give you.

YOUR UPGRADE PATH, IF YOU'RE ON 8 OR 11

Step	What to do	Why
1	Upgrade build plugins and libraries first (Lombok, Mockito, Jackson, ByteBuddy, your ORM)	Most "Java broke my app" errors are old libraries poking at JDK internals
2	Move the JDK to 17 , then 21/25 , running tests at each stop	Smaller jumps make failures easier to pin to a cause
3	If you use Spring: Boot 2 → 3 (<code>javax</code> → <code>jakarta</code>), then 3 → 4	Boot 3 requires Java 17; the namespace change is mechanical but touches everything
4	Turn on modern features gradually : records for new DTOs, switch expressions in new code	No need for a big-bang rewrite

Step	What to do	Why
5	Try virtual threads on one I/O-heavy service, with pinning monitoring (JFR <code>jdk.VirtualThreadPinned</code>)	Measure before and after; limit downstream calls explicitly

Oracle's migration guide lists removed APIs and tools, release by release. Read the sections for every version you're skipping. (JDK 25 Migration Guide) The JFR event recommendation comes from InfoQ's production checklist. (InfoQ)

In the real world: Netflix and the missing lock

Netflix moved Spring Boot 3 services (embedded Tomcat) to Java 21 virtual threads and hit **intermittent hangs**: instances stopped serving traffic, with thousands of "blank" virtual threads in their dumps. The cause was **pinning**. Virtual threads that blocked *inside* `synchronized` blocks couldn't release their carrier threads, so the few carriers filled up and everything stalled. (InfoQ on Netflix's write-up)

JEP 491 (JDK 24) fixed most of this. Virtual threads can now unmount while inside `synchronized`. (JEP 491) A 2026 InfoQ review recommends JDK 25 LTS as the first LTS with that fix. Pinning can still happen in native frames, in class initialisers, and with local file I/O on Linux. (InfoQ, Jul 2026)

Gotcha: the upgrade that "just" bumps the version

Moving from 8/11 to 17+ breaks most often **not in your code, but in old libraries**. JDK 17 strongly encapsulates JDK internals (JEP 403), so libraries that reflect into `sun.*` or private JDK fields now throw `InaccessibleObjectException`. (OpenJDK 17) The fix is to **upgrade the library** (Lombok, Mockito, older Jackson/ByteBuddy versions). Use `--add-opens` only as a temporary bridge. And if you move to Spring Boot 3+ at the same time, every `javax.*` import becomes `jakarta.*`. Oracle's migration guide lists what's been removed release by release. (JDK 25 Migration Guide)

Under the hood

- A record compiles to a `final` class extending `java.lang.Record`, with `private final` fields and generated accessors (`c.r()`, not `getR()`), `equals`, `hashCode` and `toString`. You can add methods and a **compact constructor** for validation:

```
record Money(long paise) { Money { if (paise < 0) throw new IllegalArgumentException(); } }.
```
- `sealed ... permits` is enforced by both the compiler **and** the JVM (via the `PermittedSubclasses` class-file attribute), so even reflection-generated code can't sneak in a new subtype.
- A blocked virtual thread's stack is copied to the heap as an ordinary object, and the carrier thread goes back to the pool. That's why a blocked virtual thread costs kilobytes, not a megabyte.

Try it (15 minutes, needs JDK 25+)

1. Open `jshell` and paste the sealed `Shape` example. Then add `record Triangle(double b, double h) implements Shape {}` to the `permits` list and watch `area` stop compiling until you handle it.
2. Make a file `Hello.java` containing just this, then run `java Hello.java`. No class, no `public static`, no build tool (JEP 512, Java 25):

```
void main() {  
    IO.println("Java grew up while you were away");  
}
```

WHY IT MATTERS TO YOU



Day-to-day impact



Interview value



Act-now urgency



GO DEEPER

- **Java Language Changes by Release** (Oracle docs). The official changelog of the language itself. Skim the 17 and 21 sections first.
- **Using Pattern Matching** (dev.java). The official tutorial on `instanceof`, `switch` and record patterns, with runnable examples.
- **JDK 25 Migration Guide** (Oracle). What was removed or changed between releases. Read it before any upgrade.
- **Virtual Threads after JDK 24** (InfoQ). What still pins, and a production checklist.

~10 min · Java 27 is two weeks old, Spring is in milestone season, and JDK 28 is taking shape.

Top story 1 · Java 27 is out: fewer knobs, less memory

RELEASED 15 SEP

NON-LTS

What happened. JDK 27 went GA on **15 September 2026** with **nine JEPs**. Four are final, and five are still in preview or incubation. (Oracle) (InfoQ)

Java 27 at a glance



Still preview / incubator

- Structured Concurrency (7th preview, JEP 533)
- Primitive types in patterns (5th preview, JEP 532)
- Lazy Constants (3rd preview, JEP 531)
- PEM encodings (3rd preview, JEP 538)
- Vector API (12th incubator, JEP 537)

VS



Final, ready to use

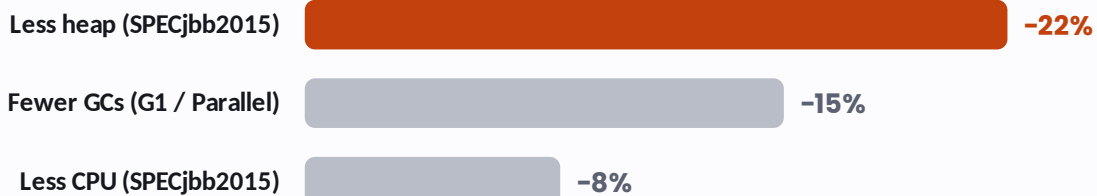
- **G1 is the default GC everywhere** (JEP 523)
- **Compact object headers on by default** (JEP 534)
- **Post-quantum hybrid key exchange for TLS 1.3** (JEP 527)
- JFR in-process data redaction (JEP 536)

Source: InfoQ, Java 27 so far.

The details that matter:

- **G1 everywhere.** Until now, on a "small" machine (roughly a single CPU or under ~1.8 GB of memory) the JVM quietly picked the **Serial** collector instead of G1. That catches people out in small containers. Now it's G1 unless you choose otherwise. (JEP 523)
- **Compact object headers by default.** Every Java object carries a header. Java 25 shipped a smaller one as an opt-in flag. Java 27 turns it on for everyone. (InfoQ) When the feature was finalised, OpenJDK reported these measurements:

What compact headers bought in OpenJDK's tests (JEP 519)



Figures from JEP 519 for one configuration each. Your app's gain depends on how many small objects it allocates. In Java 25 you opt in with `-XX:+UseCompactObjectHeaders`; in 27 it's the default.

- **Post-quantum TLS.** JEP 527 adds hybrid key exchange to TLS 1.3, pairing a classical algorithm with a quantum-resistant one. It defends against "harvest now, decrypt later" attacks, where traffic recorded today is broken by a future quantum computer. (Oracle)

Why it matters. Two of the four final features are things you get **without changing a line of code**: less memory and saner GC defaults. They will reach most teams via the next LTS, Java 29, in September 2027.

 **What to do.** Stay on **25 LTS** in production. Add a **JDK 27 job to CI** now, so the 29 upgrade is boring. If you run on 25 today, try `-XX:+UseCompactObjectHeaders` on one service in staging and compare heap usage. It's the same feature 27 now enables by default.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Interview value



Top story 2 · Spring's autumn milestones: Boot 4.2.0-M2 and friends

SPRING

MILESTONES, NOT GA

What happened. On **25 September**, Spring Boot **4.2.0-M2** shipped with 141 enhancements, fixes and upgrades. The headline items: **SSL bundle support for LDAP** (including LDAPS for the embedded server) and support for **OpenTelemetry semantic conventions**, with common OTLP endpoint configuration. ([spring.io](#)) The same week brought second milestones of Spring Framework 7.1, Spring Security 7.2, Spring Data 2026.1, Spring Batch 6.1 and Spring for Apache Kafka 4.2, which adds JSpecify nullability annotations. ([InfoQ](#))

Why it matters. Observability is the thread running through this release. A new

`management.observations.conventions` property lets you switch between Micrometer's and OpenTelemetry's naming conventions. ([InfoQ](#)) If your dashboards sit on an OpenTelemetry backend (Grafana, Honeycomb, Datadog), metric and span names will line up without custom mapping.

 **What to do.** Production stays on **Boot 4.1.x** (4.1.1 is the latest GA, with 98 fixes). ([spring.io](#)) If you're still on Boot 3.x, the 3 → 4 move is the bigger job. It's coming up in the curriculum.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



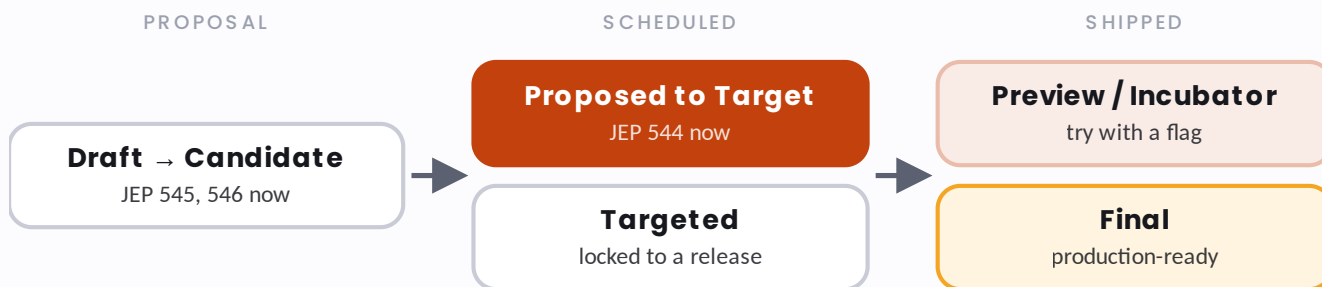
Top story 3 · The road to JDK 28 (March 2027)

WATCH

PERFORMANCE

What happened. JEP 544, **Ahead-of-Time Code Compilation**, was promoted to *Proposed to Target* for JDK 28. It aims to have optimised native code ready the moment HotSpot starts, which means faster startup and warm-up. Two ZGC JEPs (545 and 546) on adaptive heap sizing and faster startup reached *Candidate*. Early-access build 17 of JDK 28 is out. ([InfoQ](#)) InfoQ also lists **Project Valhalla's value classes (JEP 401)** among the features to watch for JDK 28. As of today they're not officially targeted. ([InfoQ](#))

How a JEP becomes a feature



Typical path. Many features spend several releases in preview (structured concurrency is on its 7th).

Why it matters. Project Leyden's AOT work goes straight at Java's oldest complaint: **slow start and warm-up**. It builds on two steps that already shipped in **JDK 25**: AOT command-line ergonomics (JEP 514) and AOT method profiling (JEP 515). (OpenJDK 25) The idea: do a "training run" of your app, save what the JVM learned (loaded classes, hot methods, profiles), and reuse it at the next start. JEP 544 goes a step further and saves *compiled native code* too, so a new pod can start near full speed instead of spending its first minutes in the slow interpreter.

For a Spring Boot service on Kubernetes, that changes the maths on **autoscaling**. New pods can take real traffic sooner, so you need less over-provisioning. Value classes, if they land, would eventually give you objects that behave like primitives in memory: no identity, flat layouts in arrays, fewer allocations. That's a big deal for anything that crunches lots of small objects (money amounts, coordinates, timestamps).

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Interview value



⚡ Quick hits



Gradle 9.8.0 supports Java 27.

It can also reuse Maven mirror repositories, and claims about 45% better performance on Windows. (InfoQ)



Maven 4.0 RC7.

The seventh release candidate of the long-awaited Maven 4. GA is expected "in the coming weeks." (InfoQ)



Quarkus 3.39.5 fixes 32 CVEs

in ecosystem dependencies, backported to the 3.33 and 3.37 lines. Worth a patch bump if you run Quarkus. (InfoQ)



Hibernate ORM 8.0 Beta3

expands support for Jakarta Persistence 4.0 milestones, including `@PostCreate` / `@PreClose` callbacks. (InfoQ)



GraalVM 25.4

brings compiler optimisation work, including better division handling. (GraalVM release notes)



Netflix open-sourced `ja`

, a command-line development tool built around the Java Module System, with module resolution, conventions and symbol search, and no heavyweight build setup. (InfoQ)



Liberica JDK 27

from BellSoft is out, with 2,542 fixes in total. (InfoQ)



Coming up.

Maven 4.0 GA, Spring Boot 4.2 GA (date not announced yet), the Jakarta EE 12 Core Profile (targeted before December), and JDK 28 in March 2027.

Revision card

- **LTS stops: 8, 11, 17, 21, 25; next is 29** (Sep 2027). Features ship every six months.
- **Preview ≠ final.** Previews need `--enable-preview`. Keep them out of production.
- **Records** = immutable data classes with free `equals` / `hashCode` / `toString`. Accessors are `x()`, not `getX()`.
- **Sealed + switch patterns** = the compiler checks every case. A new subtype breaks the build, not production.
- **Virtual threads** help I/O-bound work. Limit what's downstream (pools, APIs), not your threads.
- **Pinning** in `synchronized` was fixed in **JDK 24 (JEP 491)**. Prefer **25 LTS** for virtual threads.
- Upgrading 8/11 → 17+? **Update old reflection-heavy libraries first.** `javax` → `jakarta` comes with Boot 3+.
- **Java 27:** G1 everywhere, compact headers by default (around **-22% heap** in OpenJDK's SPECjbb test), post-quantum TLS.

Java – Beautifully Broken is researched and written by Claude every morning from OpenJDK, Oracle and Spring announcements, official docs, InfoQ and engineering blogs. Vendor numbers are labelled as claims, and every item links to its source. Want a topic sooner or a different depth? Change `java/settings.yml` or just reply. ☕